z0p: A Zero-Dependency Transparent STARK Stack in Safe Rust

nul0
Future Proofs Tech
futureproofs@proton.me

October 2026

Abstract

We present **z0p**, a complete zero-knowledge proving stack implemented from first principles in safe Rust with no dependencies beyond the compiler and the operating system. Binary fields, the BabyBear prime field, BLAKE3, Merkle trees, a parallel NTT, an arity-4 FRI polynomial commitment, four STARK systems (Fibonacci AIR, range checks, table membership, and hash-folded IVC), LogUp lookups, a minimal register-machine execution AIR, zero-knowledge masking, and a compact multilinear PCS prototype are all owned by a single crate under `#![forbid(unsafe_code)]`. The system is transparent (no trusted setup) and hash-based, hence plausibly post-quantum. Production parameters target a ≈ 100-bit query/proof-of-work schedule (rate 1/16, 40 queries, 20-bit grinding). FRI folds, STARK composition batches, `upcs`/Gemini points and betas, sumcheck rounds, and lookup identities draw challenges from a quartic extension $\mathbb{F}_{p^4}$, raising per-challenge algebraic bounds to ≈ 124 bits (see Section 6). Measured on a 16-core machine, a 1,048,576-step computation proves in 1.76 s ($\approx 1.68\,\mu$s/step), verifies in 2.2 ms, with a 203,553-byte proof. Validation combines 267 unit tests with three adversarial suites (≈ 800 hostile attempts per run, zero acceptances, zero panics), wire-exact size assertions on every benchmark, and machine-checked regression gates.

1 Introduction

Practical zero-knowledge infrastructure asks its users to trust a great deal: pairing-friendly elliptic curves, multi-party setup ceremonies, deep dependency trees, and provers whose soundness stories live in slide decks. At the same time, large-scale quantum computing threatens every discrete-logarithm and pairing-based proof system in deployment. The systems that survive that transition will be hash-based, transparently set up, small enough to audit, and tested hard enough to believe.

Contributions. z0p is, to our knowledge, the first *complete* STARK stack — fields, hash, Merkle trees, NTT, polynomial commitment, AIR systems, and zero-knowledge masking — implemented from scratch in safe Rust with zero dependencies:

- A transparent, plausibly post-quantum design: BLAKE3-256 digests and FRI Reed–Solomon commitments over BabyBear. No curves, no ceremonies.
- Production parameter sizing in one call (`Params::for_security(..., 100)`): blowup 16, queries, grinding, final layer, and cap height jointly optimized for a ≈ 100-bit query/PoW schedule. All algebraic challenges use a quartic extension field.
- Competitive scalar performance with parallelism over `std::thread` only: 1M-step proofs at ≈ 596k steps/s, millisecond verification, logarithmically growing proofs.
- Adversarial validation as a first-class artifact: every forgery class we could construct is tested continuously, including a fuzzer that once caught a genuine proof-malleability hole (since closed by binding dimensions into the transcript).

1.1 Related work

STARKs [2] and the FRI proximity test [3], strengthened by DEEP-FRI [4] and compiled via BCS-style Fiat–Shamir [1], are our direct lineage. Production STARK toolchains (notably Plonky2/Polygon Zero [8] and Risc Zero) share the BabyBear-plus-FRI engineering core but retain large dependency footprints. We differ in taking dependency-freedom and auditable minimalism as hard constraints, and in publishing adversarial test evidence alongside performance claims. Our lookups follow the LogUp logarithmic-derivative design [7]. The hash is BLAKE3 [6]; Fiat–Shamir is classical [5].

2 Preliminaries

Fields. Program words use binary fields $GF(2^8)$, $GF(2^{64})$, $GF(2^{128})$. Polynomial domains use BabyBear $\mathbb{F}_p$, $p = 2^{31} - 2^{27} + 1 = 2013265921$ with generator 31; $p - 1 = 2^{27} \cdot 15$, so 2^k-roots of unity exist for $k \leq 27$. Elements serialize as 4 canonical little-endian bytes; multiplication is Barrett reduction, inversion Fermat ($0 \mapsto 0$), batch inversion Montgomery's trick. FRI values and fold challenges live in the quartic tower $\mathbb{F}_{p^2} = \mathbb{F}_p[u]/(u^2 - 11)$, $\mathbb{F}_{p^4} = \mathbb{F}_{p^2}[v]/(v^2 - u)$; 11 is a quadratic non-residue and u a non-square; extension inversion uses the tower norm $(c + dv)^{-1} = (c - dv)/(c^2 - ud^2)$. Elements serialize as four canonical coordinates (16 bytes).

Domains. Trace domain $H = \{1, \omega, \ldots, \omega^{N-1}\}$; evaluation domain $D = \gamma \cdot \langle \Omega \rangle$ of size $M = N \cdot \beta$ (β the blowup), coset shift $\gamma = 31$; $M \leq 2^{27}$ enforced everywhere. Vanishing polynomial $Z_H(x) = x^N - 1$.

Hash and Merkle trees. Unkeyed BLAKE3-256, implemented from the specification and validated against reference vectors including chunk boundaries. Trees use $\text{leaf}(d) = H(0x00 \| d)$ and $\text{node}(l, r) = H(0x01 \| l \| r)$ for column trees (second-preimage resistant). FRI layers commit one 64-byte leaf per quad and AIR trace layers one 64-byte leaf per eight joint rows, using standard BLAKE3 for the leaf digest and the same internal-node hash. *Caps* expose an upper tree layer in the commitment so openings carry only middle path segments; FRI quad and AIR trace trees drop $\min(h, \text{depth})$ over their 64-byte-leaf counts.

Transcripts. Append-only, length-prefix-framed Fiat–Shamir transcript with a full domain-separation schedule (FRI: `fri-dims` first, then per-digest `fri-layer-root`, three `fri-fold` challenges per layer, `fri-final`, `fri-pow`, and per-query `fri-query`; per-protocol `params`, `column`, and `alpha` labels; IVC `fold/step`, `fold/trace`, six `fold/alpha` per step plus per-step `fold/gamma`; lookup and sumcheck labels analogously). Proof-of-work grinding finds the smallest nonce with d leading zero bits under the history commitment; difficulties above 256 always fail.

3 FRI polynomial commitment

`commit` encodes n coefficients to $M = n\beta$ coset evaluations (shifted NTT); `commit_evals` takes caller-supplied LDE values of declared degree $\leq$ bound $< M$. Both enforce the rate gate bound $+ 1 \leq M/2$ at prove *and* verify time. Prover-echoed dimensions are absorbed under `fri-dims` before any challenge, closing proof malleability of those bytes.

Layers are bit-reversed; each adjacent quad sits at $(x, -x, j_0 x, -j_0 x)$ for the fixed fourth root j_0, sharing x^4. Writing $P(X) = A + BX + CX^2 + DX^3$ in $Y = X^4$, each layer folds as $A + r_0 B + r_1 C + r_2 D$ with fresh challenges $r_i \in \mathbb{F}_{p^4}$ (values embed base-field elements in the first coordinate), quartering the degree; point inverses are batch-inverted once per layer and folding is multi-threaded. Proofs carry the final layer in the clear plus, per query, all four values of

every folded quad with middle path segments; each leaf hashes the canonical 64-byte encoding of its four values. The verifier replays the transcript, checks dimensions, rate, cap shapes, and PoW, verifies the final layer by divided differences against residual bound bound $\gg 2f$, then checks every Merkle path and fold relation, chaining intermediate parents forward.

Conjecture 1 (FRI soundness sizing)**.** *Under the Johnson-radius heuristic ($\delta \approx 1 - \sqrt{\rho}$), rate $\rho = 1/8$ with 12 queries gives ≈ 18 bits (bench shape); rate $\rho = 1/16$ with 40 queries plus 20-bit grinding gives an $80 + 20 = 100$-bit query/PoW schedule (production shape). Caps affect size only. These counts exclude the finite-field algebraic term. FRI folds, the AIR/range/table/fold composition batches, `upcs`/Gemini evaluation points and fold betas, sumcheck rounds, and lookup's rational-identity challenge all draw $\mathbb{F}_{p^4}$ challenges, so a nonzero degree-d identity vanishes with probability about d/p^4 per challenge. The former approximately 31-bit algebraic ceiling is gone. The production schedule remains a query/PoW formula, not an end-to-end soundness proof.*

4 AIR systems

All application protocols share one pattern: interpolate columns, extend on the LDE coset, Merkle-commit joint rows, evaluate a composition vector (geometric vanishing recurrence with Montgomery-batched denominators, bit-identical to scalar evaluation), FRI-commit it, open trace columns at FRI query positions, and re-verify the identity per query.

Fibonacci AIR. Trace $A[0] = a_0$, $B[0] = b_0$, $A[i+1] = B[i]$, $B[i+1] = A[i] + B[i]$, claim $B[N-1] = b_{\text{last}}$. With $w_{\text{last}} = \omega^{N-1}$ and $Z_{\text{trans}}(x) = (x^N - 1)/(x - w_{\text{last}})$:

$$\begin{aligned} C(x) = \alpha_0 \frac{a' - b}{Z_{\text{trans}}(x)} + \alpha_1 \frac{b' - (a+b)}{Z_{\text{trans}}(x)} + \alpha_2 \frac{a - a_0}{x - 1} \\ + \alpha_3 \frac{b - b_0}{x - 1} + \alpha_4 \frac{b - b_{\text{last}}}{x - w_{\text{last}}}. \end{aligned} \tag{1}$$

Dividing (not multiplying) by Z_{trans} is load-bearing: the quotient is low-degree iff the transition holds on every covered row.

Range checks. Value column plus $n \leq 24$ bit columns: decomposition $(V - \sum 2^j C_j)/Z_H$, per-bit boolean quotients $C_j(C_j - 1)/Z_H$, and a boundary quotient.

Table membership. $C(x) = \alpha_{\text{member}} \prod_{t \in T}(v - t)/Z_H(x) + \alpha_{\text{bound}}(v - \text{value})/(x - 1)$; small tables by design.

Folding (IVC). Chained segments share one batched FRI commitment over $\sum_s \gamma_s C_s$ with per-step challenges; one proof covers the chain.

Lookups. LogUp-style: multiplicities, β challenge, quotients $1/(\beta + w)$, $m/(\beta + t)$, rational-sum equality and four sumchecks; proving fails closed on out-of-table witnesses. A committed variant (`lookup_pcs`) commits the witness, table, multiplicities, and both quotient columns with BaseFold, binds every sumcheck final value with a point opening, and folds the total multiplicity weight into the sum-t claim with a fresh γ. At 1,024-in-1,024 with cap height 4 it proves in 367 ms, verifies in 21 ms, and carries 642,443 bytes. It is committed-input but not zero-knowledge, does not constrain columns to the base field, and pays for eight separate openings.

Program execution. The `vm` prototype proves execution of a public two-register straight-line program from a private initial state to a public output. Public one-hot selectors $\sigma_o(x)$ identify one of `Add`, `Mul`, `Swap`, `SetA`, `SetB`, or `Halt` per row; public immediate polynomials carry constants. With committed columns A, B, transition numerators T, U, and $Z_{\text{trans}}(x) = (x^n - 1)/(x - w_{\text{last}})$:

$$C(x) = \alpha_0 \frac{T(x)}{Z_{\text{trans}}(x)} + \alpha_1 \frac{U(x)}{Z_{\text{trans}}(x)} + \alpha_2 \frac{A(x) - a_{\text{last}}}{x - w_{\text{last}}} + \alpha_3 \frac{B(x) - b_{\text{last}}}{x - w_{\text{last}}}. \tag{2}$$

The final row has no outgoing transition. Composition degree is below $2n - 1$, so blowup is at least 8; production tuning uses 16. A 64-step $B16Q12C2$ plain proof proves in 1.5 ms, verifies in 0.24 ms, and carries 14,933 bytes. The plain path is not hiding; `prove_zk` masks both columns with random multiples of $x^n - 1$, preserving the trace and hiding query responses when $n_{\text{rand}} \geq Q$ (the caller sizes the final layer for the masked LDE). The program and output remain public and execution is straight-line; there is no control flow or memory.

Zero knowledge. Masked proving blinds columns with vanishing multiples from a ChaCha20 stream ($n_{\text{rand}} \geq$ queries enforced; key material zeroized on drop). Openings become witness-independent; verification is unchanged, and masked false statements still fail.

5 Encodings and parameters

Wire format (`VERSION` = 5): counts as $u32$, dimensions as $u64$, base fields as 4 canonical bytes and extension values as 16 canonical bytes ($< p$ per coordinate), digests as 32 bytes, direction bits 0/1. Lookup and sumcheck proofs use the same framing, with canonical extension values and nested versioned sumcheck bodies. The decoder fails closed on wrong versions, trailing bytes, truncation, non-canonical fields, and counts past hard caps (2^{20} queries, 64 folds, 128 path steps, 256 cap digests, 2^{27} elements, 2^{20} IVC steps) before any allocation. Validation: power-of-two domains, blowup ≥ 4 for AIR, M/final a power of four, $M \leq 2^{27}$.

6 Security analysis

Assumptions. BLAKE3 collision/preimage resistance; FRI soundness heuristic; Fiat–Shamir in the (Q)ROM. No discrete-log, pairing, or setup assumptions: Shor-inert by construction.

Enforced invariants. Bilateral rate gate; verifier-derived domains and query indices; final divided-difference check at true folded points; cap/path shape pins before hashing; `fri-dims` dimension binding; canonical-only decoding; PoW bound to history with queries sampled after.

Adversarial validation. Three layers: 267 unit tests (KATs, Horner cross-checks, parallel-vs-sequential identity); a 13-vector security suite (tampering of every query/fold/value/path/digest, 300 single-bit-flip fuzz trials, PoW forgery, rate gates both directions, statement replay); and a 10-vector red-team suite (cross-proof transplants, false statements, IVC surgery, codec-limit abuse, transcript abuse, exact rate boundaries, masked soundness, and a 500-input garbage storm that must fail closed without panicking). Standing: ≈ 800 hostile attempts per run, zero acceptances, zero panics, in debug and release.

7 Implementation and performance

Safe Rust, zero dependencies, parallelism over `std::thread` only (NTT stages $\geq 2^{16}$, folding, Merkle levels; all bit-identical). Key optimizations: Barrett field arithmetic, table-driven NTT, single-chunk hash fast paths, batched composition (geometric vanishing + Montgomery denominators), verifier on-demand domain points, Merkle caps. Measured 2026-10-20, release, medians, 16-core/31 GB, `rustc` 1.99.0:

Workload (production query/PoW schedule)	Prove	Verify	Size
AIR 1,024 steps	42 ms	0.86 ms	68,573 B
AIR 8,192 steps	54 ms	1.2 ms	101,233 B
AIR 65,536 steps	138 ms	1.7 ms	141,641 B
AIR **1,048,576** steps	**1.76 s**	**2.2 ms**	**203,553 B**
FRI poly-1024	27 ms	0.68 ms	43,174 B

Marginal cost $\approx 1.68\,\mu$s/step ($\approx$ 596k steps/s); proof sizes are deterministic and checked against the wire format every run. Verification is size-independent: each FRI domain point is derived on demand in $O(\log)$ field operations rather than from a materialized $O(\text{lde})$ layer table, so verify stays in the low milliseconds from 1,024 to 1,048,576 steps. Primitives: BLAKE3 1 MiB in 2.24 ms, BabyBear $\approx$ 150 Mops/s, NTT 2^{14} in 0.33 ms, 64k-leaf Merkle tree in 4.45 ms.

Compact multilinear PCS prototype. On 1,024 coefficients with blowup 16, 40 queries, and 20-bit grinding, the BaseFold-style prototype proves in 44 ms, verifies in 1.1 ms, and produces a 127,658-byte proof; cap height 4 gives 45 ms / 0.9 ms and 80,018 bytes. The Gemini v1 construction on the same shape takes 693 ms / 54 ms and 888,672 bytes. The prototype commits one root, reduces $f(r)$ to $c(\rho)$ with sumcheck, and uses Möbius FRI folds. Optional Merkle caps store a digest frontier per layer and hash it back to the layer root; decoder fuzzing covers both plain and capped proofs. It backs the committed lookup prototype but is not wired into the AIR stack and has no end-to-end soundness proof; treat these numbers as an engineering result, not an audit.

8 Limitations and future work

Bench defaults are fast but weak and must not be mistaken for production soundness. The table AIR is small-table by design; the committed LogUp prototype removes clear witness/table inputs but still uses eight separate BaseFold openings, lacks hiding, and does not enforce base-field column values. Vectorized hashing and the one-query recursion fragment are complete, FRI-backed univariate point and multi-point openings are in place, and Gemini plus a compact BaseFold-style prototype exist (one root, sumcheck plus Möbius FRI folds, optional Merkle caps). The prototype, committed lookup, and program-execution AIR are experimental and not wired into production AIR/STARK paths; batched openings, a hiding PCS/sumcheck, self-proving recursion, control flow, memory, instruction privacy, and a general zkVM remain roadmap items. The `vm` prototype is a public-program, private-initial-state, two-register straight-line machine with plain and masked proving; it has no memory or control flow. No external audit has yet been performed, for which the suites, gates, versioned wire format, and pinned baselines are the evidence package.

References

[1] E. Ben-Sasson, A. Chiesa, and N. Spooner. Interactive oracle proofs. In *TCC 2016*.

[2] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Report 2018/046, 2018.

[3] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Fast Reed-Solomon interactive oracle proofs of proximity. In *ICALP 2018*.

[4] E. Ben-Sasson, D. Bentov, Y. Horesh, and M. Riabzev. DEEP-FRI: Sampling outside the box improves soundness. Cryptology ePrint Archive, Report 2019/336, 2019.

[5] A. Fiat and A. Shamir. How to prove yourself: practical solutions to identification and signature problems. In *CRYPTO 1986*.

[6] J.-P. Aumasson, S. Neves, J. O'Connor, and Z. Wilcox-O'Hearn. BLAKE3: one function, fast everywhere. https://github.com/BLAKE3-team/BLAKE3, 2020.

[7] U. Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Report 2022/1530, 2022.

[8] Polygon Zero Team. Plonky2: fast recursive arguments with PLONK and FRI. https://github.com/0xPolygonZero/plonky2, 2022.